

# MoneyTracker — Documento Completo

*App iOS finanza personale per il mercato italiano · Aggiornato: luglio 2026 · Versione 3.x*

---

## 1. Visione e Posizionamento

### L'idea centrale

Una singola app, pagamento una tantum, che sostituisce carta e penna, fogli Excel, quattro app diverse, il commercialista per le cose semplici, e la testa piena di conti da fare. Apri l'app e sai esattamente dove sei, dove stai andando, e cosa puoi permetterti adesso.

### Posizionamento in una frase

*“L'unica app italiana che conosce le tue banche, capisce le tue abitudini, e ti dice esattamente cosa puoi permetterti — adesso.”*

### Differenziatori chiave

- **Italiana di design e cultura:** non una traduzione di un'app americana. Pensata per conti italiani, banche italiane, fisco italiano.
- **Completamente offline:** zero dipendenza da server, zero rischio di shutdown.
- **Una tantum:** in un mercato dominato da abbonamenti, il pagamento unico è un messaggio di fiducia.
- **Privacy radicale:** nessun dato condiviso, nessuna pubblicità, nessun tracking.
- **Design minimale:** funziona anche per chi non ha mai usato un'app di finanza.

### Modello di business

**Attuale:** distribuzione TestFlight con Apple ID personale (certificato 7 giorni).

**Al lancio App Store:** acquisto una tantum €4,99 — accesso completo, nessun abbonamento, nessuna pubblicità.

**Futuro (monetizzazione aggiuntiva):** Piano Premium opzionale €1,99/mese per connessione bancaria automatica, AI Financial Coach, sync Supabase cross-device, Modalità Coppia/Famiglia.

---

## 2. Stack Tecnico e Architettura

| Componente     | Tecnologia                                                                              |
|----------------|-----------------------------------------------------------------------------------------|
| Linguaggio     | Swift 6.0 (strict concurrency)                                                          |
| UI Framework   | SwiftUI                                                                                 |
| Persistenza    | SwiftData (@Model, ModelContext, FetchDescriptor)                                       |
| Concorrenza    | Swift 6 @MainActor, structured concurrency con Task                                     |
| Notifiche      | UserNotifications<br>(UNCalendarNotificationTrigger, UNTimeIntervalNotificationTrigger) |
| Shortcuts/Siri | AppIntents framework (AppIntent, AppShortcutsProvider)                                  |
| Logging        | OSLog (Logger.persistence, .recurring, .classifier)                                     |
| Sicurezza DB   | NSFileProtectionComplete su SQLite WAL/SHM                                              |
| Cloud Sync     | Supabase (SDK integrato, in sviluppo)                                                   |
| iOS target     | iOS 17+                                                                                 |
| Localizzazione | 7 lingue (IT, EN, ES, FR, DE, ZH-Hans, JA)                                              |
| Architettura   | MVVM implicita, computed properties<br>SwiftUI-native                                   |
| Xcode project  | PBXFileSystemSynchronizedRootGroup (auto-scan cartelle)                                 |

### Pattern architetturali

- **Singleton:** NotificationManager.shared, FormatterCache (entrambi @MainActor)
- **NSCache:** CategoryClassifier.resultCache per risultati di classificazione
- **Transferable:** CSVFile con DataRepresentation (nessun file temp su disco)
- **AppStorage:** persistenza preferenze UI (hideBalance, monthlyReportEnabled, filtri)
- **UserDefaults:** soglie saldo (keyed by account UUID, evita migration SwiftData)

## 3. Struttura del Progetto

```

MoneyTracker/
├── MoneyTrackerApp.swift      ← Source principale
├── seed categorie             ← Entry point, container SwiftData,
├── Auth/
│   ├── AppLockGate.swift      ← Blocco biometrico app
│   └── BiometricAuthenticating.swift

```

- ├─ LoginView.swift ← Login Supabase
- ├─ SignUpView.swift
- ├─ SupabaseManager.swift ← Client Supabase, auth, sync
- ├─ Debug/
  - ├─ DemoDataSeeder.swift ← Dati demo per testing
- ├─ Domain/
  - ├─ AccountBalanceCache.swift ← Cache saldi calcolati
  - ├─ AuditLogger.swift
  - ├─ CategoryClassifying.swift ← Protocollo classificazione
  - ├─ KeywordCategoryClassifier.swift ← Classificatore keyword on-
- device
  - ├─ NotificationManager.swift ← Notifiche locali budget/saldo
  - ├─ NotificationScheduling.swift
  - ├─ RecurringTransactionEngine.swift ← Engine ricorrenze
- automatiche
  - ├─ Export/
    - ├─ PDFReportGenerator.swift ← Generazione PDF mensile
  - ├─ Formatting/
    - ├─ CurrencyFormatting.swift ← Estensioni Double/Date
    - ├─ FormatterCache.swift ← Singleton formatter costosi
  - ├─ Intents/
    - ├─ AddExpenseIntent.swift ← Apple Shortcuts integration
  - ├─ Models/
    - ├─ Account.swift
    - ├─ Budget.swift
    - ├─ Category.swift
    - ├─ Goal.swift
    - ├─ Transaction.swift
  - ├─ OnboardingTour/
    - ├─ TabBarFrameCapture.swift
    - ├─ TourHighlight.swift
    - ├─ TourManager.swift
    - ├─ TourOverlayView.swift
    - ├─ TourStep.swift
  - ├─ Persistence/
    - ├─ Logging.swift
    - ├─ ModelContextSaving.swift
    - ├─ SchemaMigration.swift
  - ├─ Resources/
    - ├─ CategoryKeywords.json ← Keywords per classificazione
- automatica
  - ├─ Sync/
    - ├─ SyncService.swift ← Sync Supabase (in sviluppo)
  - ├─ Views/
    - ├─ AccountsView.swift
    - ├─ AddTransactionView.swift
    - ├─ AddTransferView.swift
    - ├─ AuditLogView.swift
    - ├─ BudgetView.swift
    - ├─ CategoryManagementView.swift
    - ├─ ContentView.swift ← Tab bar principale

|  |  |                               |                                                |
|--|--|-------------------------------|------------------------------------------------|
|  |  | DashboardView.swift           | ← Home                                         |
|  |  | GoalsView.swift               |                                                |
|  |  | PianificaView.swift           |                                                |
|  |  | SettingsView.swift            |                                                |
|  |  | StatisticsView.swift          |                                                |
|  |  | Theme.swift                   | ← Design system completo                       |
|  |  | TransactionsView.swift        |                                                |
|  |  | [*.lproj]                     | ← Localizzazioni                               |
|  |  | (it/en/de/fr/es/ja/zh-Hans)   |                                                |
|  |  | Assets.xcassets/              |                                                |
|  |  | MoneyTrackerTests/            | ← Unit test (XCTest)                           |
|  |  | MoneyTrackerUITests/          | ← UI test (XCUITest, simula uso reale)         |
|  |  | UITestSupport.swift           | ← Classe base, helper navigazione/FAB          |
|  |  | TransactionFlowUITests.swift  | ← Aggiungi/modifica/elimina movimento          |
|  |  | AccountFlowUITests.swift      | ← Aggiungi/modifica/elimina conto              |
|  |  | CategoryFlowUITests.swift     | ← Aggiungi/modifica/elimina categoria          |
|  |  | NavigationStressUITests.swift | ← Uso intensivo: tab switching, sheet ripetuti |
|  |  | docs/                         |                                                |
|  |  | MoneyTracker.pdf              | ← Questo documento (unico riferimento)         |
|  |  | graphify-out/                 | ← Knowledge graph del progetto                 |

**Automazione notturna** (fuori dal repo, in ~/Desktop/NightTestApp/): uno script (run\_nightly\_tests.sh) lanciato ogni notte alle 4:00 da un LaunchAgent (~/.Library/LaunchAgents/com.moneytracker.nightlytests.plist) esegue gli XCUITest su simulatore; se falliscono, invoca Claude Code headless per analizzare e correggere il codice, riprova fino a 5 volte, scrive report\_notturmo.md nella root del progetto e apre una Pull Request con le correzioni. Vedi sezione 10 per i dettagli del funzionamento.

## 4. Modelli Dati

### Account

Rappresenta un conto bancario, carta, portafoglio o investimento.

| Campo          | Tipo   | Descrizione                                         |
|----------------|--------|-----------------------------------------------------|
| id             | UUID   | Identificatore univoco                              |
| name           | String | Nome del conto (es. “Carta Unicredit”)              |
| type           | String | Tipo: conto corrente, carta, contanti, investimento |
| initialBalance | Double | Saldo iniziale inserito dall’utente                 |
| isArchived     | Bool   | Se archiviato, non appare nelle liste attive        |

| Campo               | Tipo   | Descrizione                                      |
|---------------------|--------|--------------------------------------------------|
| isExcludedFromTotal | Bool   | Se escluso, non concorre al saldo totale in home |
| colorHex            | String | Colore personalizzato                            |

**Computed:** currentBalance (solo tx isDone == true), futureBalance (tutte le tx), plannedDelta (differenza futuro–attuale).

**Relazioni:** transactions: [Transaction] (cascade delete), budgets: [Budget] (cascade delete).

## Transaction

| Campo              | Tipo     | Descrizione                                          |
|--------------------|----------|------------------------------------------------------|
| id                 | UUID     | Identificatore univoco                               |
| name               | String   | Descrizione (es. “Affitto”, “Stipendio”)             |
| amount             | Double   | Importo                                              |
| date               | Date     | Data della transazione                               |
| type               | String   | “expense” o “income”                                 |
| isFixed            | Bool     | Spesa/entrata fissa ricorrente                       |
| isDone             | Bool     | true = già effettuata, false = pianificata           |
| isTransfer         | Bool     | Fa parte di un trasferimento tra conti               |
| transferGroupId    | String   | UUID condiviso tra le due gambe di un trasferimento  |
| category           | String   | Categoria assegnata                                  |
| recurringFrequency | String   | Frequenza: vuoto / daily / weekly / monthly / yearly |
| account            | Account? | Relazione con il conto                               |

## Budget

| Campo    | Tipo     | Descrizione                                      |
|----------|----------|--------------------------------------------------|
| id       | UUID     | Identificatore univoco                           |
| category | String   | Categoria a cui si applica                       |
| limit    | Double   | Limite mensile in valuta                         |
| account  | Account? | Opzionale: limita il budget a un conto specifico |

## Goal (Obiettivo di risparmio)

| Campo         | Tipo   | Descrizione                   |
|---------------|--------|-------------------------------|
| id            | UUID   | Identificatore univoco        |
| name          | String | Nome obiettivo                |
| emoji         | String | Emoji selezionata dall'utente |
| targetAmount  | Double | Importo target                |
| currentAmount | Double | Importo risparmiato finora    |
| hasDeadline   | Bool   | Se ha una scadenza            |
| deadline      | Date?  | Data scadenza opzionale       |

## Category

| Campo     | Tipo   | Descrizione                               |
|-----------|--------|-------------------------------------------|
| id        | UUID   | Identificatore univoco                    |
| name      | String | Nome categoria                            |
| iconName  | String | Nome SF Symbol                            |
| isDefault | Bool   | Le categorie default non sono eliminabili |

## FormatterCache (performance)

Singleton centralizzato per formatter costosi. Ricrea i formatter solo quando cambia la valuta o la locale — elimina N allocazioni NumberFormatter/DateFormatter per ogni render di ogni row.

- `currencyFormatter()` — NumberFormatter currency riutilizzato
- `monthYear` — DateFormatter("MMMM yyyy") singleton
- `dayMonth` — DateFormatter("d MMM") singleton
- `shortMonth` — DateFormatter("MMM") singleton per grafici

## 5. Design System (Theme.swift)

### Palette colori semantica

| Token    | Descrizione                                      |
|----------|--------------------------------------------------|
| DS.ink   | Testo principale (nero in light, bianco in dark) |
| DS.paper | Sfondo principale                                |
| DS.fog   | Sfondo secondario (grigio chiarissimo)           |
| DS.smoke | Testo secondario, label, placeholder             |

## Spacing

DS.Space.xs / s / m / l / xl / xxl — sistema a scala fissa per coerenza visiva.

## Componenti riutilizzabili

| Componente                               | Descrizione                                         |
|------------------------------------------|-----------------------------------------------------|
| HeroAmount(amount:size:hidden:)          | Importo in grande. Con hidden == true mostra “•••”  |
| SectionLabel(text:)                      | Intestazione sezione in uppercase con letterspacing |
| PrimaryButton(title:action:)             | Bottone principale nero/bianco                      |
| GhostButton(title:action:)               | Bottone secondario testo-only                       |
| ThinDivider()                            | Separatore sottile in DS.fog                        |
| DSTransactionRow(transaction:)           | Riga transazione standard con VoiceOver completo    |
| MonthBar                                 | Barra grafico mensile con label via FormatterCache  |
| EmptyStateView(icon:title:subtitle:cta:) | Empty state coerente con CTA opzionale              |

## Modalità chiara e scura

Full support Light/Dark mode nativo SwiftUI, senza override manuali.

## 6. Funzionalità Implementate

### Home (DashboardView)

- **Saldo Attuale Hero** — totale conti inclusi in grande (HeroAmount, ~56pt bold)
- **Chips conti** — riga orizzontale scrollabile con nome + saldo. Pulsante ☰ apre pannello selezione/ordinamento (HomeAccountOrderSheet). Ordine persistito via @AppStorage
- **Nascondi saldo** — pulsante occhio: HeroAmount e chips mostrano “•••”. Stato persistito via @AppStorage("hideBalance")
- **Saldo Atteso** — appare solo se ci sono movimenti pianificati. Mostra saldo futuro + delta (▲/▼)
- **Ultimi Movimenti** — collapsabile. Ultime 4 transazioni non fisse. Swipe destra = Modifica, sinistra = Elimina
- **Toolbar:** pulsante tema (sinistra) + occhio + ingranaggio (destra)

### Movimenti (TransactionsView)

- Lista completa per data (più recenti in cima)

- **Filtri a due righe:** Tutti · Fisse · Trasf. · Uscite · Entrate + tutte le categorie. Filtri combinabili
- **Filtri avanzati** persistiti via `@AppStorage` (importo min/max, date)
- Statistiche mensili in header con cambio mese (freccie avanti/indietro)
- **Swipe destra:** “Fatto/Pianificato” per spese fisse e trasferimenti; “Modifica” per le altre
- **Swipe sinistra:** Elimina (singola o entrambe le gambe del trasferimento via `transferGroupId`)

## Aggiunta Transazione (AddTransactionView)

- Campi: nome (con categorizzazione automatica on-device), importo, tipo (uscita/entrata), categoria, conto, data
- Toggle “Già effettuato” (`isDone`), Toggle “Spesa fissa” (`isFixed`)
- Attivando “Spesa fissa” disattiva automaticamente “Già effettuato”
- Categorizzazione automatica: keyword matching on-device senza rete. L’utente può sovrascrivere manualmente
- Keyboard navigation: importo → nome → `.done` (auto-save)
- Reset form corretto al riutilizzo (`load()` resetta tutto quando `editing == nil`)

## Trasferimenti (AddTransferView)

- Seleziona conto “Da” e “A”, inserisci importo
- Toggle “Già effettuato” (default: attivo)
- Genera due transazioni collegate con stesso `transferGroupId`
- Bottone dinamico: “Trasferisci” o “Pianifica” in base allo stato

## Conti (AccountsView)

- Lista: prima conti inclusi nel totale, poi esclusi (badge “escluso”)
- **Swipe:** destra = Modifica, sinistra = Elimina (con haptic)
- **Context menu** (long press): Modifica / Escludi-Includi / Archivia / Elimina
- **Conti archiviati:** sezione separata, opacità 35%. Context menu: Ripristina / Elimina definitivamente
- Hero: saldo attuale + futuro dei conti inclusi
- Nuovo conto non escluso → aggiunto automaticamente in home

## Budget (BudgetView)

- Budget per categoria con limite mensile in valuta
- Barra di avanzamento (verde/rosso per soglia superata)
- Filtro opzionale per conto specifico
- Storico budget (ultimi mesi) via `BudgetHistorySheet`
- Notifiche locali quando ci si avvicina o supera il limite

## Obiettivi (GoalsView)

- Nome + emoji, importo target, importo attuale, deadline opzionale
- Barra di avanzamento e percentuale completamento
- VoiceOver completo: nome, %, importi, deadline, stato completato

## Statistiche (StatisticsView)

- Grafico a torta per categoria (solo tx isDone == true)
- Trend mensile a barre (entrate vs uscite ultimi 6 mesi)
- Confronto annuale: mese corrente vs stesso mese anno scorso
- Esportazione PDF mensile (via PDFReportGenerator)
- Esportazione CSV (via Transferable, nessun file temp su disco)
- Loading indicator animato durante generazione PDF/CSV

## Impostazioni (SettingsView)

- Cambio valuta (€, \$, £, ¥, ecc.) — si aggiorna dinamicamente ovunque via `Double.currencySymbol`
- Cambio lingua (7 lingue)
- Selezione tema
- Toggle “Report mensile PDF” — notifica locale il 1° di ogni mese alle 10:00
- Avviso saldo basso per conto (persistito in UserDefaults)

## Biometria (Auth/)

- AppLockGate: blocco app con Face ID / Touch ID via LocalAuthentication
- Login/SignUp Supabase (in sviluppo per sync cloud)

## Shortcuts Apple (Intents/)

- Azione “Aggiungi spesa” disponibile nell’app Comandi di iOS
- Aggiungibile come widget su schermata home o di blocco
- Inserisci nome e importo → salvato direttamente nel database senza aprire l’app
- try? safe init del ModelContainer (no crash in caso di store non disponibile)

## Onboarding Tour (OnboardingTour/)

- TourManager: gestione step, completamento, avanzamento
- TourOverlayView: overlay con card modale (ModalCard, StepCard)
- TourHighlight: sistema di highlight con anchor key
- TabBarFrameCapture: cattura frame tab bar per posizionamento overlay

## Ricorrenze Automatiche

- Campo “Ripeti ogni” in AddTransactionView: mai / giornaliero / settimanale / mensile / annuale
- `Transaction.recurringFrequency`: String nel modello
- `RecurringTransactionEngine`: al lancio app genera automaticamente nuove occorrenze per tx scadute
- Le nuove tx vengono create come `isDone = false` (pianificate)

## Localizzazione

7 lingue complete (IT, EN, ES, FR, DE, ZH-Hans, JA). 340 chiavi per file. Accessibility labels localizzate con `String(localized:)` (non string interpolation italiana).

## Haptic Feedback

- Swipe “Modifica”: haptic soft
- Swipe “Elimina”: haptic medium
- Tutti i bottoni distruttivi: haptic medium

## 7. Audit Qualità (Score: 7.2 / 10)

| Area                       | Voto          |
|----------------------------|---------------|
| Architettura               | 7/10          |
| Qualità codice / Bug       | 7.5/10        |
| Performance                | 7/10          |
| Standards / Manutenibilità | 7.5/10        |
| Sicurezza                  | 8/10          |
| <b>Totale</b>              | <b>7.2/10</b> |

## Cosa funziona bene

- Swift 6 strict concurrency rispettata ovunque — zero data races
- OSLog strutturato con subsystem/category separati
- `NSFileProtectionComplete` sul database `SwiftData`
- Formula injection prevention nel CSV export
- Design system coerente (DS namespace, `EmptyStateView`, `HeroAmount`)
- `LazyVStack` per liste lunghe (no List con altezza fissa)
- Notifiche con ID idempotenti (remove-then-add)
- VoiceOver completo su tutti i componenti principali

## Bug risolti (sessioni precedenti)

| ID     | Problema                                                                        | Stato                                   |
|--------|---------------------------------------------------------------------------------|-----------------------------------------|
| BUG-01 | DashboardView List + frame fisso<br>→ testo troncato con Dynamic Type           | ✅ Fixato → LazyVStack                   |
| BUG-02 | GoalsView.load() → “1000.0”<br>invece di “1000”                                 | ✅ Fixato → truncatingRemainder          |
| BUG-03 | CategoryStat.id UUID instabile<br>→ rompeva diffing SwiftUI                     | ✅ Fixato → computed var stabile su name |
| BUG-04 | print() di debug in<br>TabBarFrameCapture → console<br>noise su layout hot-path | ✅ Fixato → rimossi 7 print              |

| ID      | Problema                                                    | Stato                                           |
|---------|-------------------------------------------------------------|-------------------------------------------------|
| PERF-01 | Allocazione NumberFormatter/DateFormatter per ogni render   | ✅ Fixato → FormatterCache singleton             |
| SEC-01  | try! su ModelContainer in Shortcuts → crash dell'estensione | ✅ Fixato → try? + guard con errore user-visible |
| i18n-01 | Sezione Siri non tradotta in nessuna lingua                 | ✅ Fixato → 10 chiavi aggiunte a tutti i 6 file  |
| i18n-02 | Accessibility labels in italiano hardcoded                  | ✅ Fixato → String(localized:) ovunque           |
| PERF-02 | € hardcoded in AddAccountView e AddTransferView             | ✅ Fixato → Double.currencySymbol                |

### Issue residue (non critiche)

| ID       | Problema                                                               | Priorità  |
|----------|------------------------------------------------------------------------|-----------|
| PERF-R01 | Account.currentBalance O(n_tx) — scala male con >5000 tx               | Alta — v2 |
| PERF-R02 | @Query senza fetchLimit in DashboardView                               | Media     |
| SMELL-01 | CategoryManagementView sezione vuota → bare Text                       | Bassa     |
| SMELL-02 | AddGoalView, AddBudgetView senza @FocusState                           | Media     |
| SEC-02   | NotificationManager.isEnabled non verifica autorizzazione OS effettiva | Media     |
| SEC-04   | CheckBalanceIntent biometric policy rimossa (API iOS 26 cambiata)      | Bassa     |

## 8. Roadmap

### Livello 2 — Necessario per essere competitivi

| Feature                           | Stato                                    |
|-----------------------------------|------------------------------------------|
| Tour onboarding / discoverability | ⚙ In sviluppo (OnboardingTour/ presente) |

| Feature                                                    | Stato                                                          |
|------------------------------------------------------------|----------------------------------------------------------------|
| Widget schermata Home e Lock Screen (WidgetKit + AppGroup) | <input type="checkbox"/>                                       |
| Ricorrenze automatiche                                     | <input checked="" type="checkbox"/> Implementato               |
| Apple Watch App (watchOS target + WatchConnectivity)       | <input type="checkbox"/>                                       |
| Face ID / Touch ID blocco app                              | <input checked="" type="checkbox"/> Implementato (AppLockGate) |

### Livello 3 — Feature WOW (differenziazione totale)

| Feature                                | Descrizione                                                                                          |
|----------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Connessione bancaria automatica</b> | GoCardless/Nordigen API (gratuito EU) — Intesa, UniCredit, Fineco, BancoBPM, Poste                   |
| <b>“Me lo posso permettere?”</b>       | Shortcut conversazionale on-device: “scarpe 150€” → risposta immediata con contesto budget           |
| <b>AI Financial Coach</b>              | Report mensile automatico il 1° del mese, on-device. Opzionale: sintesi via Claude Haiku API         |
| <b>Tracker abbonamenti</b>             | Rilevamento automatico pagamenti ricorrenti, alert per abbonamenti non usati                         |
| <b>Punteggio Salute Finanziaria</b>    | Score 0–100 settimanale su 5 dimensioni (budget, obiettivi, fondo emergenza, spese fisse, risparmio) |
| <b>Sync cross-device (Supabase)</b>    | iPhone + iPad, offline-first, TLS + AES-256, last-write-wins                                         |
| <b>Modalità Coppia/Famiglia</b>        | Database condiviso via CloudKit o Supabase                                                           |
| <b>Split spese</b>                     | Divisione conto con contatti iPhone + iMessage/WhatsApp automatico                                   |
| <b>Calendario flusso di cassa</b>      | Vista calendario con proiezione saldo giorno per giorno                                              |
| <b>Tracciatore Patrimonio Netto</b>    | Attivi – Passivi (banca + investimenti + immobili – mutuo + debiti)                                  |
| <b>Feature fiscali italiane</b>        | Bollette tracker, F24 tracker, Detrazioni fiscali 19%, 730 helper, TFR tracker                       |
| <b>Modalità “Stipendio → Zero”</b>     | Zero-based budgeting guidato (metodologia YNAB)                                                      |

### Livello 4 — Futuro

Investimenti automatici (Directa/Fineco API), Crypto wallet tracker, FIRE Calculator, “Boomerang” confronto periodo anno scorso, Spesa emotiva, Marketplace partner opt-in, API pubblica.

## 9. Setup e Installazione

### Requisiti

- Mac con macOS 13+
- Xcode (gratuito dall'App Store del Mac, ~7 GB)
- iPhone con iOS 17+
- Cavo USB (prima volta)

### Installazione da sorgente

1. Apri MoneyTracker.xcodeproj in Xcode
2. Signing & Capabilities → seleziona il tuo Team (Apple ID)
3. Collega iPhone al Mac con cavo → “Autorizza” sul telefono
4. Seleziona il tuo iPhone come dispositivo target
5. Premi ► (Play)
6. Prima volta: Impostazioni iPhone → Privacy e sicurezza → App per sviluppatori → Autorizza il tuo Apple ID

### Configurare il banner Shortcuts

1. App Comandi sul iPhone → “+” → “Aggiungi azione” → cerca “Money Tracker”
2. Seleziona “Aggiungi spesa”
3. ... → “Aggiungi a schermata Home” o “Aggiungi a schermata di blocco”
4. Tocca l'icona → banner in cima → inserisci nome e importo → salvato senza aprire l'app

### Note

- **Certificato:** con Apple ID gratuito scade ogni 7 giorni. Ricollegare il telefono e premere Play per rinnovarlo.
- **Dati al sicuro:** SwiftData non cancella i dati durante gli aggiornamenti.
- **Trasferimento:** richiede almeno 2 conti non archiviati.

### Errori comuni

| Problema                               | Soluzione                                                  |
|----------------------------------------|------------------------------------------------------------|
| “Signing failed”                       | Xcode → Signing & Capabilities → Team → seleziona Apple ID |
| App non si apre                        | Impostazioni → Privacy → App per sviluppatori → Autorizza  |
| Certificato scaduto (7 giorni)         | Ricollega iPhone, premi Play in Xcode                      |
| “Personal teams do not support iCloud” | Non aggiungere iCloud/Push con Apple ID gratuito           |

## 10. Changelog

### v3.x (luglio 2026) — XCUITest e automazione notturna

- Nuovo target **MoneyTrackerUITests** (XCUITest) nel progetto Xcode, creato via script Ruby (xcodeproj gem) sul project.pbxproj
- 12 test UI in 4 classi: flusso movimenti, conti, categorie, e uno stress test di navigazione (tab switching rapido, apertura/chiusura ripetuta di fogli, aggiunta/eliminazione multipla)
- Aggiunto hook `MoneyTrackerApp.isUITesting` (`--uitesting` come launch argument): bypassa login Supabase, Face ID e tour di onboarding, forza la modalità demo con dati deterministici — nessuna chiamata di rete durante i test
- Aggiunti `accessibilityIdentifier` su tutti gli elementi interattivi chiave (campi, bottoni, swipe actions) in `AddTransactionView`, `AccountsView`, `CategoryManagementView`, `SettingsView`, `TransactionsView`, `ContentView` (FAB), `BudgetView`, `GoalsView`
- `TestPlan.xctestplan` aggiornato per includere il nuovo target
- **Routine notturna**: script `run_nightly_tests.sh` + `LaunchAgent` `com.moneytracker.nightlytests.plist` (fuori dal repo, in `~/Desktop/NightTestApp/`) — esegue i test ogni notte alle 4:00, corregge automaticamente il codice con Claude Code headless in caso di fallimento (fino a 5 tentativi), scrive `report_notturmo.md` e apre una PR

### v3.x (luglio 2026) — Riorganizzazione e documentazione

- Riorganizzata struttura cartella: `Views/`, `Domain/`, `Models/`, `Auth/`, `Export/`, `Intents/`, `Debug/`, `Persistence/`, `Formatting/`, `OnboardingTour/`, `Sync/`
- Eliminati file residui (`.Rhistory`, `worktree` stantio)
- Tutta la documentazione consolidata in questo unico file

### v3 (giugno–luglio 2026) — Enterprise audit e bug fix

#### 43 miglioramenti su 6 aree:

- **FormatterCache**: eliminata allocazione `NumberFormatter/DateFormatter` per ogni render (hot path)
- **DashboardView**: List fissa → `LazyVStack` (Dynamic Type fix)
- **GoalsView**: `EmptyStateView`, VoiceOver, bug `load()` con “1000.0”
- **BudgetView**: `EmptyStateView`, VoiceOver su `BudgetRow`
- **AddTransactionView**: keyboard navigation con `@FocusState`
- **AddTransferView**: `.submitLabel(.done)` su importo
- **AddExpenseIntent**: `try!` → `try? safe init` (SEC-01 critico)
- **StatisticsView**: `CategoryStat.id` stabile, `shortMonthFormatted` via `FormatterCache`
- **TransactionsView**: `filteredSnapshot` per accesso centralizzato
- **NotificationManager**: alert saldo basso per singolo conto
- **SettingsView**: loading indicator CSV/PDF, toggle report mensile, animazioni spring
- **i18n**: sezione Siri localizzata in 6 lingue, accessibility labels con `String(localized:)`
- **QA**: rimossi 7 `print()` di debug da `TabBarFrameCapture` e `TourOverlayView`

## v2 (primavera 2026) — Feature complete

- Conti multipli con saldo attuale / saldo atteso
- Trasferimenti tra conti (inclusi pianificati)
- Budget per categoria con filtro per conto
- Obiettivi di risparmio
- Shortcut Apple (banner senza aprire l'app)
- Multi-lingua 7 lingue
- Statistiche mensili con grafici
- Esportazione CSV e PDF
- Ricorrenze automatiche (recurringFrequency)
- Biometria (Face ID / Touch ID)
- Onboarding tour (in sviluppo)

## v1 (inverno 2025) — Base

- Progetto iniziale: transazioni, categorie, saldo singolo conto
- 

# 11. Workflow Checkpoint

Ogni volta che si fa un checkpoint significativo:

1. **Aggiorna questo documento** (docs/MoneyTracker.md) con le modifiche apportate
  2. **Rigenera il PDF:** `cd /Users/matteo/Desktop/MoneyTracker && pandoc docs/MoneyTracker.md -o docs/MoneyTracker.pdf --pdf-engine=weasyprint`
  3. **Commit e push:** `git add -A && git commit -m "checkpoint: [descrizione]" && git push`
  4. **Aggiorna il grafo:** `/graphify --update` in Claude per aggiornare graphify-out/
- 

*Documento unico — aggiornato ad ogni checkpoint · Generato con Claude (Anthropic)*